

A Chatroom, Independent Attempts, and One Agent on Three Tasks

[Maintainer name and affiliation — to be filled before submission]

Corresponding author: [maintainer email]

Abstract

We compare a single agent, a three-seat chatroom, and selection from independent attempts on machine-scored tasks at roughly matched mean spend. All benchmark results are dated measurements on recorded subscription accounts, of `claude-sonnet-5` under the `sonnet` alias in Studies 1 and 2 and of `claude-opus-5-5` in Study 3, not estimates for other models or deployments. In Study 1, the chatroom passed both interpreter tasks on every seed, against 61/80 single-agent passes across their shared task family. The vote arm passed 75/80; direct tests did not distinguish it from the chatroom. On a numeric-formatting task, one agent passed 14/40, the chatroom 6/40, and the vote 7/40. The lower chatroom rate was significant under the pre-registered test but not a later Fisher test. Voting selected the common wrong implementation even though correct attempts were often available. The vote prediction succeeded on one interpreter task but failed its stated significance clause on the other.

The same recorded alias and settings yielded sharply different thinking-token distributions across subscription accounts. Study 2 mostly ran on the long-thinking account, leaving its pre-registered short-thinking primary comparisons essentially unobserved. Its small, exploratory long-thinking comparisons distinguished no arm from one agent. The comparison shows why serving-account provenance belongs with each run’s outcome.

A post-hoc audit of 260 structured verification entries across 36 rooms found that most reported only builds, existing tests or a scripted smoke client, about a fifth an added probe or before/after check, and none a run of agents on a changed build. In the first auditor’s coding of four rooms, re-runs were the largest review category and had the lowest defect-flagged share among categories with a stated checking method. A passing report of the author’s own command satisfies the verification gate.

Study 3 gave pools of 20 independent items with hidden tests and a 30-minute deadline to one agent, to three agents each holding a fixed third, and to rooms of three and fifteen seats. In both completed pools the three-seat room landed 5 more items than the split, against the prediction that they would not differ. Splitting never landed more than one agent, and fifteen seats never more than three, while cost per passing item rose with the number of seats. The pools were close to the ceiling, and a third pool was stopped after two runs.

1 Introduction

A chatroom can give several language-model agents a chance to challenge and revise one another’s work. It also buys more attempts at the task. To assess what the conversation contributes, the relevant comparison is with independent attempts chosen without seeing the answer, at a similar cost. This paper makes that comparison on a small, deliberately difficult task suite, then shows how a change in the serving account limited a later, broader study.

The measured system is `agent-chatroom-mcp`. Studies 1 and 2 used `claude-sonnet-5` under the `sonnet` alias between 18 and 20 September 2026; Study 3 used `claude-opus-5-5` on 23 and

24 September 2026. The subscription account active at run time is reconstructed from a sanitized switch log. Pass rates, costs, and which task was near ceiling belong to those dates, model and accounts. They are not predictions for newer models. The study method and the need to retain serving provenance do carry forward; the observed minority-versus-majority pattern is a hypothesis to test elsewhere.

Study 1 used seeds 101–140 on three fixed tasks, comparing one agent (A), a three-seat chatroom (C), and independent attempts with oracle-free selection (K). The third task asks for exact numeric formatting. All outcomes came from a machine oracle, not from the agents’ own verdicts.

Study 2 used seeds 501–520 and added a higher-effort single agent (AH) and a builder-reviewer pair (B). Its pre-registered primary comparison targeted short thinking. The run stopped with missing cells after most work was served by the long-thinking account. The resulting long-thinking comparisons were specified after data had been seen and remain exploratory.

Study 3 asked a throughput question: with 20 independent, testable items from one repository and 30 minutes, does one agent, three agents splitting the items, a three-seat room or a fifteen-seat room land the most? Items and their hidden tests were written by an AI curator.

A review audit, added after Studies 1 and 2, examines what one seat’s check of another seat’s work consisted of, across 36 rooms run with a mix of models (Section 4.5).

The paper makes five narrow points. In Study 1, direct tests did not distinguish the chatroom from independent selection, even though both could outperform one agent on the interpreter family and both had lower observed success on printf. Across recorded accounts, thinking tokens differed under the same alias and settings, with one long-thinking single agent near ceiling on the measured interpreter task. Study 2 cannot answer its primary question: the intended short-thinking stratum was almost absent. Finally, a passing report of the author’s own command satisfies the system’s verification gate; most verification entries reported re-running existing checks, and none reported running agents on a changed build. In Study 3, a three-seat room landed more items than split work in both completed pools, while splitting did not beat one agent and fifteen seats did not beat three.

2 Evaluated system

The evaluated build of `agent-chatroom-mcp` gave each chatroom seat an independent opening before shared discussion. Seats could propose and amend one answer, challenge a quoted span, and vote on the current proposal. Agreement votes had to quote the proposal; the room used a supermajority rule that required unanimity with three active seats. A verification gate required a machine-parseable verdict from a connection other than the proposer, and the hub assigned that reviewer. The verify gate and reviewer assignment were present before both studies. Kick, replace and heartbeat changes dated 22 September were later additions and were not evaluated here. Study 1 chatroom cells record hub entry hash 66f9d135...; run artifacts carry the revision used for each cell.

Arm A used one agent on the same public task brief. Arm K ran fresh, non-communicating A-style attempts and selected one answer without access to the hidden oracle. For the two interpreter tasks, it chose the most frequent normalized final answer, breaking a tie by sub-seed. For printf, it ran candidate programs on a fixed public-specification probe set and chose the implementation with the greatest execution agreement. Arm B in Study 2 passed one builder’s output to a reviewer for possible revision; AH used one agent at higher effort. Thus B was run, while Study 1 has no B cell.

3 Method

Tasks and scoring. Study 1’s tasks were admitted after a single-agent pilot screened about 45 designs for a discriminating pass rate. The admitted tasks each contain one known trap. `stamp-interpreter` and `stamp-2` ask for programs implementing the same invented-language scoping rule; they are two fixed instances of one family, not independent evidence from two domains. `bench-printf-format` asks for exact formatting where a natural implementation using the host runtime’s shortest representation disagrees with the specified output. The same hidden, machine-checked fixtures scored every arm. The seeds vary attempts on fixed task instances, rather than sampling new tasks. The task suite therefore supports comparisons on these traps, not an estimate across software engineering work.

The pilot targeted single-agent pass rates between 0.3 and 0.7, inclusive. The interpreter admission pilots used ten seeds each; the retained, raw-backed `printf v1` pilot used fifteen seeds; `v2` used ten, and five earlier `v1` seeds lacked retained artifacts. Two versions of the `printf` task were piloted, and the one used here was chosen after both pilots’ results had been seen, a post-data selection.

Study 1 design. For each task, A and C ran on seeds 101–140, interleaved per seed. A’s budget ceiling was tied to C’s realized spend; matching did not force the same realized spend. K was separately pre-registered after A and C results but before K ran, on the same seeds and tasks. Its number of attempts was chosen from the earlier mean A and C costs, with each attempt run afresh. The selector had no oracle input. The fixed K group sizes were ten attempts for `stamp-interpreter`, eight for `stamp-2`, and seven for `printf`. The per-attempt cap left room for an ordinary A-style attempt; total group cost included every attempt even if it was killed or not selected. The matching target was mean realized C spend, not identical spend on every seed.

K ran from 13:22 to 15:33 UTC, before the account switch recorded at 16:38 UTC; an earlier shift-time estimate was superseded by that switch log. The comparison between K and C still spans separate collection windows. K exceeded its paired C spend on 19, 10, and 9 of the 40 seeds for the three tasks, despite matching at the mean.

The original A–C protocol specified a two-proportion z-test, using Fisher’s exact test for small cells, separately by task. After the data, the two interpreter tasks were pooled as a family and A–C was retested with Fisher’s exact test and Holm correction across the two families. We report that revised family result alongside the original per-task tests. The K pre-registration fixed six two-sided Fisher comparisons, K against A and C on each task, with Holm correction across all six. Pooled K comparisons on the interpreter family are exploratory and do not replace those registered tests.

The common seed labels put arms on the same task brief, but the attempts are separate model runs; the Fisher calculations do not treat them as paired observations.

Study 2 design and missingness. Study 2 specified five arms on seeds 501–520: A, AH, B, K and C. Its primary tests used the short-thinking stratum. The account switcher was not known to the harness at launch. A sanitized log later associated each run with its active account; mixed-account seeds were separated from within-regime comparisons. The run stopped at 190/200 cells, leaving ten absent. `Printf` seed 501 is kept in descriptive totals but excluded from inference after a documented cap deviation. One `printf` chatroom run has unknown cost. The long-thinking tests were added after much of that stratum had been observed, and use 19 seeds per arm on `stamp-interpreter` and 15 on `printf`. They use Fisher tests with Holm correction across the registered comparison family of fourteen comparisons, but their timing makes them exploratory. Strata

condition on A’s realized post-run thinking, rather than an independent pre-run measurement; its sentinel also cannot rule out a change later in the same seed.

Account measurement. The account analysis joins single-seat run intervals to switch times and uses only runs with thinking telemetry for the thinking-token comparison. Long thinking was defined in the Study 2 pre-registration as at least 4,000 thinking tokens. Five of 1,947 eligible runs straddled a switch and were excluded. Output-token distributions give a second descriptive account comparison, but output across a three-seat room is not directly comparable to a single seat. Neither the active account nor effort was randomized. The client records the association, not the server-side reason for it.

Study 3 design. Study 3 asked a different question from Studies 1 and 2: given 20 independent, testable items from one repository and 30 minutes, how many items does each setup land? The setups were one seat with all 20 items (Solo); three seats each given a fixed third of the items and no chat, whose branches the harness merges afterwards in a fixed order, keeping the earlier branch’s version on a conflict (Split); a room of three workers and a verifier that organises itself from the list and requires verification (Room3); and the same room with fourteen workers (Room15). Every seat ran `claude-opus-5-5` from the brief alone, in its own git worktree of a fresh repository holding only the pool’s base tree, and was told the deadline. Rooms were scored on the integration branch they declared.

Each item was written by an AI curator as a hidden acceptance test plus a reference fix, and entered a pool only if its test failed three times out of three at the base and passed three times out of three with the fix. Briefs state the symptom, the required behaviour and the interface the test checks, never the cause, the files to change or the approach; from the second pool on, a leak reviewer compared each brief with its fix and a blind fairness reviewer, who never saw the fix, checked that the test asked only for what the brief states. No two items’ reference fixes change the same function. Three pools were planned in a fixed order: this system (pool 1), a browser game (pool 2) and a larger product repository frozen at one commit, dependencies included (pool 3). After pool 1 reached the ceiling, later items had to take one agent longer: pool 2’s replacement items and every pool 3 item needed a reference fix touching at least two source files and adding at least 50 lines, and the project’s own test suite had to pass with each fix. Before pools 2 and 3 were locked, a practice Solo run acted as a ceiling gate: a pool locked if it landed 16 or fewer, otherwise its smallest items were swapped for larger reserves and the gate was run once more.

Each setup ran twice per pool, one run at a time, in a seeded random order. The primary measure is the number of items whose hidden test passes at the setup’s final head. Secondary measures are cost from the command-line client’s own usage records, cost per passing item, items attempted, merge conflicts and whether the project’s suite still passes at the final head. Any access to the hidden tests would void a run. The pre-registration fixed three predictions (Split lands more than Solo; Room3 and Split do not differ; Room15 lands more than Room3) and a direction rule: a difference counts when two setups’ means differ by 3 or more items, in the same direction in at least two of the three pools. With two repeats per setup, no significance test is run.

4 Results

4.1 Study 1: conversation and independent selection

Table 1 gives the A–C family comparison; Figures 1 to 3 show the same Study 1 results as charts. Across the two interpreter programs, A passed 61/80 and C passed 80/80. The family-pooled

Table 1: Study 1, A versus C by task family.

Family	A pass/ <i>n</i>	C pass/ <i>n</i>	Fisher <i>p</i> (raw)	Holm <i>p</i> (<i>m</i> = 2)	Sig.
interpreter (stamp-interpreter + stamp-2)	61/80	80/80	0.000001	0.000002	yes
printf (bench-printf-format)	14/40	6/40	0.069165	0.069165	no

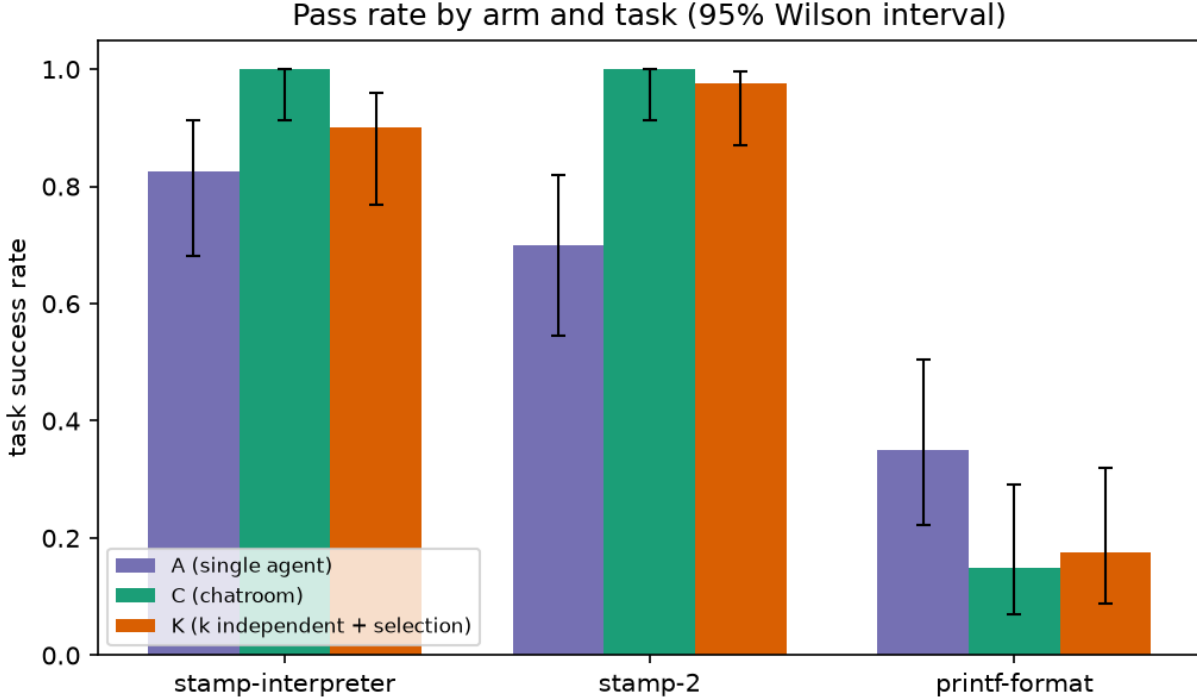


Figure 1: Study 1 task success by arm and task, with Wilson intervals at the 95 per cent level; 40 seeds per cell, short-thinking account, 19 September 2026. The chatroom (C) passed 40/40 on both interpreter tasks; on printf both aggregation arms fell below one agent (A).

Fisher test, adopted after the runs, gives $p = 0.000001$ before Holm adjustment. The original pre-registered per-task tests also rejected A–C on each interpreter program; the pooled result is a revised analysis, not the pre-registered test. K passed 36/40 on **stamp-interpreter** and 39/40 on **stamp-2**, or 75/80 pooled. Its pooled comparison with A ($p = 0.0033$) is exploratory. The registered K tests show a narrower result: K beat A on **stamp-2** (Holm $p = 0.009$), but was not distinguished from A on **stamp-interpreter** (Holm $p = 1.000$). The pre-registered prediction said K would beat A on both. That clause failed.

Direct K–C tests did not distinguish the arms on any Study 1 task: every Holm-adjusted p was at least 0.578. Pooling the interpreter tasks gives 75/80 for K and 80/80 for C, with an exploratory Fisher $p = 0.059$. Neither result establishes equality. In particular, a significant C–A result and a nonsignificant K–A result on **stamp-interpreter** are not themselves a test of C against K. The direct comparison is the relevant test, and the available sample leaves a small difference unresolved.

On **bench-printf-format**, A passed 14/40, C 6/40, and K 7/40. The pre-registered A–C z-test gave $p = 0.039$; the later family-based Fisher test gave $p = 0.069$, so the significance decision depends on the stated test. K was not distinguished from A (Holm $p = 0.578$) or C. The observed

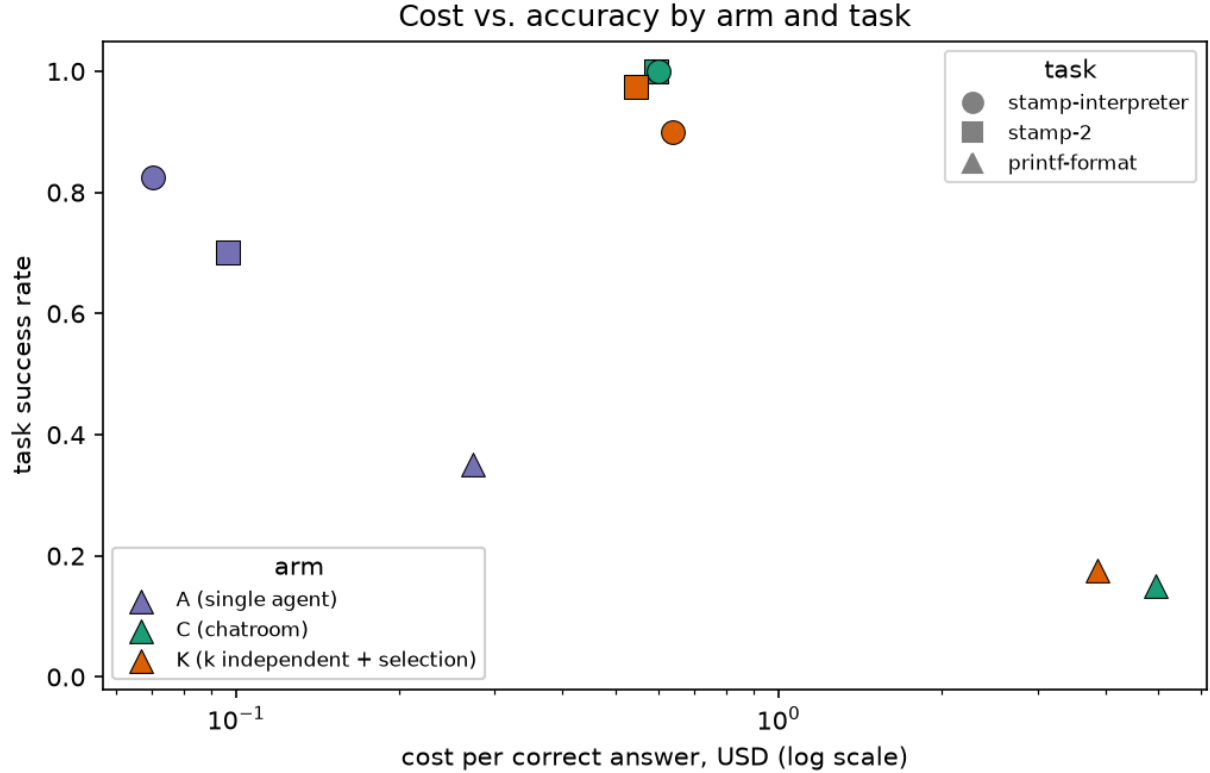


Figure 2: Study 1 cost per correct answer (log scale, API list price) against success rate, one point per task and arm. One agent was the cheapest per correct answer on every task; the chatroom and the vote cost about the same as each other on the interpreter tasks, and the vote cost less on printf (\$3.87 against \$4.97).

rates on this task are lower for both aggregation arms; they do not support a general assertion that either method lowers success on other tasks.

The independent-attempt evidence explains K’s printf outcome more specifically. In a fixed pool of 40 A attempts, all 14 passing implementations agreed on every one of 9,650 public-specification probes, and all 26 failing implementations also agreed with one another. Thus execution agreement identified the larger wrong cluster. In the real K groups, at least one correct attempt was available in 37/40, but the oracle-free selector submitted a correct one in only 7/40. The availability count is an oracle ceiling, not a deployable arm. The C artifacts retain pass or fail, but not the failing implementations; the same modal-error explanation for C is an association, not an observed C selection mechanism.

The interpreter answers show the opposite population pattern in Study 1. For **stamp-interpreter**, the seven A failures among 40 were identical; for **stamp-2**, 11 of the 12 failures were identical and one differed. Here the correct answer was the majority of A answers in the observed pool. The room summaries add diagnostic context: at least 26 of 40 **stamp-interpreter** rooms and 14 of 40 **stamp-2** rooms reported an erroneous opening followed by another seat’s correction. These are manual lower bounds from final seat summaries, without retained raw room events. They do not identify a causal chatroom mechanic.

The extra computation had a substantial price in these runs. C’s cost per correct answer relative

Arm K mechanism: does agreement track correctness?

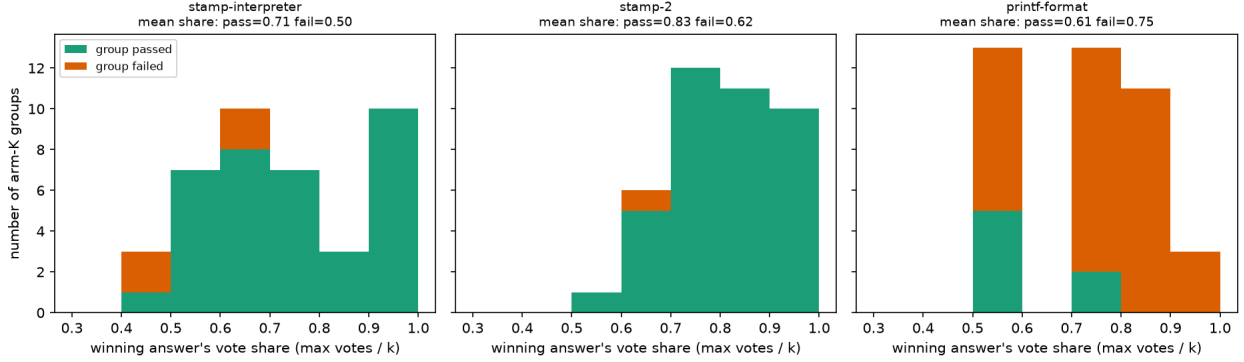


Figure 3: Study 1 vote arm: in each group, the winning answer’s share of the votes, split by whether the selected attempt passed. On printf, strong agreement goes with failure, because the failing implementation was the larger cluster of independent attempts.

Table 2: Recorded subscription account and run-level thinking or output tokens. Generated from the per-run records joined to the account-switch log.

Active account	Runs	Long-thinking runs	Median thinking tokens	Min	Max
Account 1	40	0/40	968	463	1295
Account 3	11	0/11	716	595	1316
Account 4	385	384/385	7709	3742	31604

Task family	Active account	Runs	Median output tokens	Min	Max
bench-printf-format	Account 1	337	4645	3789	6595
bench-printf-format	Account 3	26	4225	3726	5181
bench-printf-format	Account 4	152	25144	16212	43700
stamp-interpreter	Account 1	590	1716	1357	2232
stamp-interpreter	Account 4	313	7310	4494	16896

Single-seat runs (arm A, arm AH, and the constituent attempts of arm K), joined by start time to a sanitized extract of the maintainer’s account-switcher log (switch times and account slot numbers only). 5 of 1947 runs straddle a switch and are left out. Top: runs that recorded thinking tokens; long thinking means at least 4000 thinking tokens, the threshold fixed in the Study 2 pre-registration. Bottom: output tokens, which every run recorded, for the two Study 2 task families. Account numbers are slots in the switcher, not identifiers.

to A was 8.56 times on **stamp-interpreter**, 6.16 times on **stamp-2**, and 18.17 times on **printf**. K and C cost respectively \$0.637 and \$0.602 per correct answer on **stamp-interpreter**, \$0.545 and \$0.595 on **stamp-2**, and \$3.87 and \$4.97 on **printf**. These are list-price calculations under the recorded accounts and include all K attempts, including those not selected.

4.2 Account-associated thinking

Table 2 joins single-seat run times to the sanitized switch log. Among runs with thinking telemetry, account 4 had 384/385 at or above 4,000 thinking tokens. Accounts 1 and 3 had none among 51 combined runs. Median thinking was 7,709 tokens on account 4, against 968 and 716 on accounts 1 and 3. Within each of the two task families shown, account 4’s output-token range lay entirely above those of the other measured accounts. These are strong associations under the same alias and settings, not evidence that the client can identify what the provider changed.

The association matters for interpreting accuracy. Short-thinking single-agent measurements on

Table 3: Study 2 descriptive outcomes by task, stratum and arm. Inference excludes mixed-account seeds and printf seed 501; the eligible long-thinking comparisons use fewer seeds than some descriptive rows.

Descriptive rows retain printf seed 501; it is excluded from both primary and exploratory comparisons.						
Family	Stratum	A	AH	B	K	C
stamp-interpreter	long thinking	18/19	19/19	19/19	19/19	19/19
stamp-interpreter	mixed accounts	1/1	1/1	1/1	1/1	1/1
bench-printf-format	short thinking	0/1	0/1	1/1	0/1	0/1
bench-printf-format	long thinking	12/16	12/16	11/16	13/16	10/16
bench-printf-format	mixed accounts	0/1	0/1	1/1	1/1	0/1

stamp-interpreter came from several cohorts: the admission pilot, the Study 1 A grid, constituent K attempts, a cap control, and the window control. Their estimates ranged from 0.475 to 0.825. The Study 1 grid’s 33/40 was therefore one estimate within an observed span, not a stable model parameter. The separate long-thinking window-control cohort had 78/80 successful constituent single attempts. Study 2’s long-thinking A arm passed 18/19 on that task and cost \$0.136 per correct answer. These are different cohorts, not a randomized account comparison; their gap does not show that either account or effort caused an accuracy change. The source of the variation among short-thinking estimates is also unidentified. **stamp-2** was not measured under long thinking, so the long-thinking interpreter result is confined to **stamp-interpreter**.

4.3 Study 2: the primary stratum was nearly absent

Study 2’s pre-registered primary used short-thinking seeds. It obtained no eligible short-thinking **stamp-interpreter** seed and only one printf seed. The interpreter’s apparent short-thinking seed crossed accounts between arms and was excluded from within-regime inference. One printf seed also crossed accounts. Table 3 retains those rows descriptively; the printf seed with a cap deviation is likewise descriptive only. The missing ten cells and excluded seeds leave the registered primary comparisons essentially unobserved.

Unlaunched cells are absent from the denominators; launched cells with non-pass outcomes count as failures. Unknown cost is not treated as zero.

Within the exploratory long-thinking stratum, the interpreter comparisons use 19 seeds per arm. A passed 18/19; AH, B, K and C each passed 19/19. On printf, the inferential sample has 15 seeds per arm: A and AH passed 11/15, B and C 10/15, and K 12/15. Every Holm-adjusted comparison against A had $p = 1.000$. That test result means no arm was distinguished in these small samples, not that their effects are equal or that the primary prediction was confirmed. On the interpreter, B cost \$0.269 per correct answer and C \$1.407, with both passing 19/19. The pair’s observed cost was about a fifth of the chatroom’s. The pre-registered cost prediction concerned short thinking; this exploratory long-thinking inequality does not confirm it.

4.4 Study 3: a small room landed more than split work; a large one did not

In both completed pools, Room3 landed 5 more items than Split: means of 20 against 15 in pool 1 and 19 against 14 in pool 2. The prediction that the two would not differ is contradicted. Split never landed more than Solo (15 against 20 in pool 1, 14 against 14 in pool 2), and Room15 never

Table 4: Study 3: items passing (of 20) in each counted run, with the minutes the run took, and cost per passing item over both repeats. Pool 3 was stopped after two runs.

Pool	Setup	Repeat 1 (min)	Repeat 2 (min)	Cost per passing item
Pool 1 (this system)	Solo	20 (4.8)	20 (7.8)	\$0.11
	Split	15 (7.2)	15 (11.9)	\$0.16
	Room3	20 (9.1)	20 (9.0)	\$0.33
	Room15	20 (8.8)	20 (9.6)	\$1.14
Pool 2 (browser game)	Solo	20 (25.5)	8 (12.6)	\$0.29
	Split	14 (9.8)	14 (9.4)	\$0.41
	Room3	19 (14.1)	19 (13.9)	\$0.71
	Room15	20 (15.1)	19 (22.3)	\$1.76
Pool 3 (product repository)	Solo	not run	not run	–
	Split	not run	20 (11.1)	\$0.44
	Room3	not run	not run	–
	Room15	not run	20 (26.8)	\$3.41

more than Room3 (20 against 20, then 19.5 against 19), so those two predictions are not supported. Pool 3 was stopped after two of its eight runs and gives no direction (Table 4). No run was voided for access to hidden tests; one pool 1 run was voided for a harness defect, in which one seat’s process-matching command stopped fourteen of fifteen seats, and was rerun.

Cost per passing item rose with the number of seats in both completed pools: \$0.11, \$0.16, \$0.33 and \$1.14 for Solo, Split, Room3 and Room15 in pool 1, and \$0.29, \$0.41, \$0.71 and \$1.76 in pool 2. In pool 3, Split and Room15 each landed all 20 items, in 11.1 and 26.8 minutes, for \$8.71 and \$68.14.

Observations, not measures. Split lost items to merges, not to unfinished work: in pool 1 the same five items were lost in both repeats to one conflict in a shared file, and in pool 2 each repeat had two conflicts. Every seat in every counted run ended its own turn before the deadline. Solo’s second pool 2 run stopped at 12.6 minutes having attempted 9 items, so Solo’s mean there averages two different behaviours of the same setup. In both pool 3 practice runs, every item the seat attempted passed; the second landed 13 only because the seat stopped early.

4.5 How seats check each other

This subsection is an exploratory audit added after Studies 1 and 2; it was not pre-registered. It asks what one seat’s check of another seat’s work consisted of. It covers every verification write whose first line is a JSON object in the hub’s logs of rooms and sub-rooms created before 10:00 UTC on 23 September 2026: 260 writes from 36 rooms created between 17 and 23 September, of which 24 worked on this system’s code, research or documentation, 10 on browser games and 2 on another codebase. These rooms used a mix of Claude, Codex and DeepSeek models; they are not the benchmark runs of Studies 1 and 2.

What the gate asks for. In a room that requires verification, a proposal can pass only once a qualifying verification entry exists. The entry must use a verification key not marked partial and begin with a parseable head: string fields for the proposal, the command, its working directory

Table 5: The strongest check each structured verification entry reports, by the kind of project its room worked on.

Strongest check the entry reports	Hub	Game	Other	All
Build, type-check or existing tests only	31	95	0	126
Scripted smoke client, model spawning off	43	0	0	43
Read code, docs or history; ran nothing	6	1	3	10
Added check: own probe or before/after run	42	13	0	55
The built application in a real browser	0	26	0	26
Model-driven agents on the changed build	0	0	0	0
Not enough detail to tell	0	0	0	0
Rooms and sub-rooms	24	10	2	36

All 260 verification entries with a structured head, classified by the strongest check the entry’s text reports. Two AI coders classified every entry independently from its full text (agreement 256/260, Cohen’s $\kappa=0.98$); an adjudicator settled the 4 disagreements, and 3 labels were corrected after an independent review. Classes describe what the entry says was run, not observed execution; an added check is either a probe the verifier wrote or a before/after run that may reuse existing tests.

Hub: rooms on this system’s code, research or documentation; Game: rooms building a browser game.

and the tail of its output, a numeric exit code of 0, and a commit if one is given. It must come from a seat other than the proposer, on a different connection, and be no older than the proposal’s current text; while a reviewer assigned by the hub remains in the room, only that reviewer’s entry counts. The gate does not ask what the command exercised or whether it differs from the author’s own check, and it cannot establish that the command ran: the head is a structured report. An entry reporting a failure cannot satisfy the gate; only a separate path for executable challenges lets a verifier rule a challenger’s command invalid at a nonzero exit code, with a stated reason. When verification is required and the board holds no verification entry at all, the hub refuses a proposal and asks the author to post the command they ran, adding that “someone else must then run it”. When an open proposal still needs a blocking challenge and a seat has not yet voted, that seat may receive a hint calling the proposal “open and untested” and asking for its weakest claim in one sentence. A passing report of the author’s own command therefore satisfies the gate. The gate supplies nothing like the held-out tests applied during SWE-bench evaluation [3].

What verification entries report. Table 5 and Figure 4a classify each entry by the strongest check its text reports. Two same-model AI coders labelled all 260 entries independently from their full text and agreed on 256 (Cohen’s $\kappa = 0.98$); an adjudicator resolved the other four. Agreement does not establish correctness: an independent review of 48 entries found three labels that both coders had got wrong, and they are corrected here. Of the 260 entries, 241 parse under the hub’s own head parser, 238 report exit code 0 and 128 name the commit they checked; 110 were written by a seat other than the proposal’s known author, 11 by the author, and for 139 the author could not be resolved. Most entries report re-running existing checks: 126 only builds, type checks or existing tests, and 43 the scripted smoke client, which drives a local hub with model spawning disabled; 10 report only reading code or history. A substantial minority report more. Fifty-five report an added check, either a probe the verifier wrote or a before/after run such as a regression test run against the unfixed code, which may reuse existing tests; 26, all in game rooms, report driving the built application in a real browser. No entry reports its writer running model-driven agents on a changed build, though that does not establish that none ran. Among the 110 entries by a seat other than the known proposal author, 34/110 report an added check and 11/110 a browser run.

What review acts found. In four of these rooms, two on this system and two game rooms, one AI auditor catalogued review acts from the room log and each seat’s own transcript and classified

Table 6: Review acts by method in the four audited rooms, as classified by the first auditor, and the acts it marked as finding a defect.

Review method	Hub A	Hub B	Game 1	Game 2	All	Found a defect
Re-ran author’s test/build	44	12	59	39	154	39
Read the diff only	20	18	21	14	73	42
Own check	13	7	9	9	38	25
Used it as users would	3	12	8	14	37	14
Checked commit contents	6	1	3	11	21	11
No stated check	3	2	3	0	8	0
All review acts	89	52	103	87	331	131

The first AI auditor’s classification of review acts (one seat checking another’s work) in two rooms on this system and two browser-game rooms, from the room logs and the seats’ own transcripts, and the acts it marked as finding a defect. Labels include mixed methods and disputed findings; a second auditor found omitted acts, disputed classifications, repeated reports and self-checks counted as peer review (Section 4.5).

each by method. A second auditor re-counted each room: it found acts the first had missed, disputed some classifications, and flagged repeated reports and self-checks (Table 6, Figure 4b). In the first auditor’s coding, re-running the author’s test, suite or build was the largest category, 154 of 331 acts, and had the lowest defect-flagged share among categories with a stated method: 39/154, against 42/73 for reading the diff and 25/38 for a check the author had not supplied. These are associations. They do not show why a reviewer chose a method or which method produced a finding, and a reviewer may write its own check because it already suspects a problem.

In the first room on this system, the author of a benchmark pilot ran model agents on an integration build containing other seats’ changes; the auditors concluded that the pilot did not meaningfully test draft sealing and found no reviewer inspecting its logs. In the second, neither auditor found a reviewer running model agents on a changed build. Across both, the acts the first auditor counted as using the system “as its users would” were scripted smoke checks, observations of the unchanged live hub, and one changed-build launcher check that stopped at a refusal. The second auditor estimated about 35 requests about test evidence in the first room, mostly for re-runnable commands or scope caveats; requests led to the pilot on the integration build, and a later discussion ended in a decision not to re-run it on the final head. In the game rooms, non-author seats did open the game, at least 8 and 15 times by the auditors’ counts. The first auditor listed 12 and 10 items that lacked relevant non-author exercise in the game, some grouping several changes; the re-counts corrected several items, including a pre-merge browser check of one game’s residents change. That check covered night scenes only, and the owner reported the change failing in daylight about six minutes after its merge was announced.

Reading. Most structured verification entries reported builds, existing tests or the smoke client, and in the separate review-act audit re-runs were the largest category, 154 of 331 acts; a substantial minority of checks added a probe, a before/after run or a browser session. No verification entry reported running agents on a changed build of a system whose claims are about how agents behave. A passing report of the author’s own command satisfies the gate. The audit includes no controlled comparison of verification rules, so it cannot say what a different rule would have caught.

5 What this does not show

The planned vote-versus-chatroom comparison under one stable serving condition was never run at adequate power.

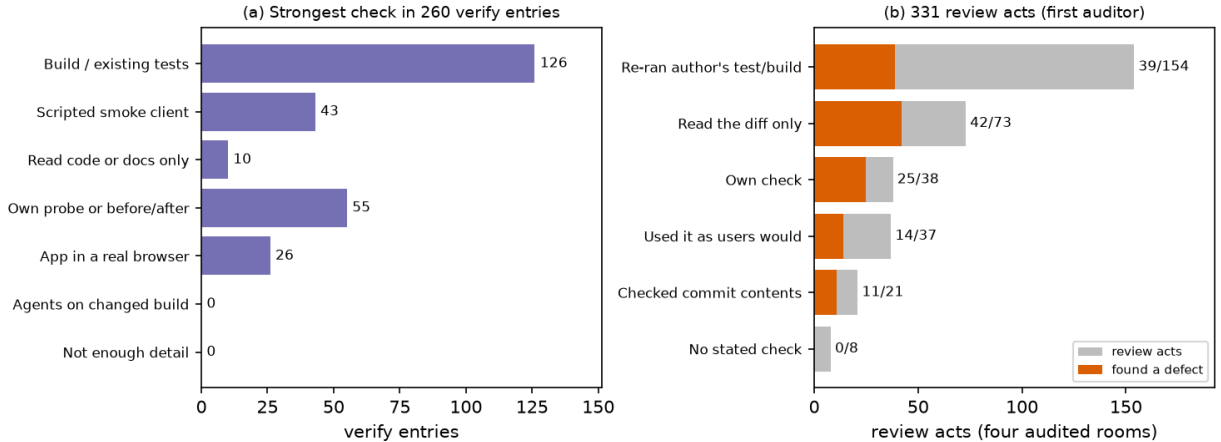


Figure 4: (a) The strongest check reported by each of the 260 structured verification entries. (b) The first auditor’s classification of review acts in the four audited rooms; the orange share is the acts it marked as finding a defect.

The maintainer’s working pipeline—prompt, prior-art agent, plan, build, and devkit review—has never been compared with one agent, a builder-reviewer pair, or a room. The marginal value of review by a non-author was not identified either. B ran in Study 2 and in a small pilot, but the Study 2 primary was essentially absent. A separate build-suite diagnostic reached a 9/9 ceiling in every cell.

The bundled chatroom build was compared with other arms, but its individual mechanics were not ablated. Regression tests show that gates and assignments operate as coded; one bundled-build trial is not a causal estimate for a gate. No effect size is assigned here to blind openings, challenge, amended proposals, quote-checked votes, reviewer assignment, or verification.

The review audit (Section 4.5) is post hoc and was not pre-registered. Its census covers structured verification entries only, and its classes come from AI coders reading what each entry says was run, not from observed execution; the two coders share a model, and an independent review of 48 entries found three labels both had got wrong, which are corrected. Its second measure rests on AI auditors whose counts moved on re-count and whose labels include mixed methods and disputed findings. Its rooms mix models and projects and were run by one operator. It describes recorded verification practice; it does not establish what a different rule would have caught.

Other engineering observations are outside the task-success comparison. Held waits were measured at 115.6 million versus 53.6 million tokens in one room each on different tasks, so that contrast is observational. OpenRouter sticky-session caching was mock-tested only. Checkpoint trimming has no answer-quality test. None of these results supports a claim about the measured A, B, C, or K accuracy differences.

Study 3 has two repeats per setup and two completed pools, so its direction count is a coarse summary, not a test. Its pools were close to the ceiling. In pool 1 every setup except Split landed all 20 items, pool 2’s rooms landed 19 or 20, and both pool 3 runs landed 20. Items passing therefore cannot separate Solo from the rooms or the two rooms from each other; time and cost can, and they are reported rather than tested. The ceiling gate counted items a practice Solo run passed, but a seat that stops early passes fewer items without the items being harder: in both pool 3 practice runs every attempted item passed, and the second still cleared the gate. Items were written and reviewed by AI agents. All runs shared one machine, whose test runner allowed only a few test

runs at once, so larger rooms waited longer for tests. Pool 3 was stopped after two of its eight runs, at the operator’s decision, once it was clear the pool could not separate the setups.

6 Related work

Multi-agent debate provides the immediate setting for this comparison. Du et al. [1] study debate as a route to better answers, while Smit et al. [6] examine how debate strategies compare with simpler baselines. Those papers motivate asking whether a conversational protocol earns its additional cost. Our measured question is narrower: on three fixed machine-scored tasks, did this recorded chatroom differ from a vote over fresh attempts?

Arm K’s interpreter selector is a form of self-consistency: independently generated answers are normalized and the most frequent is submitted [8]. Its code selector instead uses execution agreement among candidate programs, following the type of signal studied by Shi et al. [5]. In the observed printf pool, agreement favored the common wrong implementation. That is why the candidate-availability ceiling and the submitted-answer rate must be reported separately. The result describes the selector as used here; it is not a comparison among alternative selectors.

The chatroom’s independent openings, challenge, proposal and verification steps make it tempting to attribute corrected answers to a particular interaction. Work on divergent thinking and premature consensus [4; 9] gives reasons to inspect those interactions. Work on minority answers in multi-agent debate [2] likewise frames the wrong-cluster observation as a testable question rather than an established general rule.

Finally, the hidden machine oracle keeps the agents’ own consensus separate from correctness. Research on language-model judges [10; 7] concerns a different measurement route; none of the pass rates here uses such a judge. Our main measurement complication arose earlier, at model serving: the same recorded alias and settings accompanied different thinking-token distributions on different active accounts. The account log and token telemetry therefore belong alongside accuracy and cost in any attempt to reproduce these particular comparisons.

A Pre-registration, deviations and reproducibility

What was fixed in advance. The Study 1 A–C comparison and its per-task test were specified before any Study 1 run. The vote arm (K), its group sizes, selectors, per-attempt caps and six-test Holm family were specified after the A and C results but before any K run. Study 2’s five arms, seeds, rotation of arm order within each seed, thinking-token threshold, fourteen-test Holm family and predictions were specified before any Study 2 cell existed. Study 3’s setups, measures, predictions, direction rule and pool order were specified before any Study 3 run; each pool’s items, split and run order were hash-locked before that pool ran.

Recorded changes. Before the Study 1 grid, the original task suite was replaced following screening, and the long-brief task was retired. The admission rule was tightened after earlier candidates proved near ceiling; the below-0.3 exclusion category was added after observing printf v2. The retained printf pilot and post-data version selection are described in Section 3. The original fifteen-seed design with sequential stopping was replaced by forty seeds per task. Dollar and wall-clock ceilings derived from each paired C run replaced token/turn matching; seeds served as bookkeeping labels without controlling sampling or seat assignment. The room size was fixed at three, the default deadline increased from 300 to 900 seconds, turn counts came directly from response JSON, and deadline-killed runs were classified as timeouts and made retryable. K’s caps,

selectors and six-test family were settled before K collection. The later A–C analysis revision is described in Section 3. The auxiliary same-window control was stopped after three runs for cost and resumed at reduced scope.

In Study 2, the printf deadline bound on its first A cell. Caps were then raised for A, AH and every K attempt, and the printf block’s spending ceiling increased from 74 to 240 USD. The original timed-out A cell was retained separately and rerun; seed 501’s inferential exclusion is described in Section 3. Killed cells with partial usage from every seat subsequently counted toward the launch budget at list-price estimates multiplied by 1.5; this did not recover their actual cost. An outcome-blind rule invalidated nine quota-refused cells and the operator-killed B cell at printf seed 517; all were rerun, retaining their known spend. Remaining launches were restricted to account 4. Account-log stratification, mixed-seed exclusions and the post-data long-thinking tests are described in Sections 3 and 4.3. The switch-log extract was later refreshed to correct resumed-cell assignments. The maintainer stopped after printf seed 518 because further cells were judged unlikely to change the observed lack of differences; this was an outcome-informed stop without a pre-specified stopping rule.

In Study 3, pool 1’s second Room15 run was voided after one seat’s process-matching command stopped fourteen seats whose prompts it matched; prompts moved to standard input and the run was repeated in the same position. After pool 1, each run got its own fresh repository, so no seat could see earlier runs’ branches, and the harder-item floor and ceiling gate were added for pools 2 and 3. Pool 2’s first practice run landed 18, so eight larger items replaced the eight smallest before its second practice run landed 13. Pool 3 was drawn from a product repository still under active development: its source and dependencies were frozen at one commit outside that repository, the repository’s other work sessions were asked to leave the chosen items unbuilt, and three candidates were withdrawn before validation because a session was already rebuilding that area. A harness time limit that killed every full-suite check while it waited for a free test slot was raised before any pool 3 validation result was used, and all candidates were validated again. After its first practice run landed 18, pool 3’s two smallest items were swapped for its two spares. The remaining runs were then paced by subscription headroom, and the pool was stopped after two runs.

How the numbers were produced. Every table and figure is generated by scripts from per-run result records; none is edited by hand, and a regeneration check fails if a regenerated table or figure differs from the one included. Account assignment joins each run’s start and end times to a sanitized extract of the account switcher’s log that holds only switch times and account slot numbers. The manual room-correction tallies are described in Section 4.1. The automated audit checks extracted prose decimals, fractions and percentages for numerical matches in generated outputs and listed records, allowing rounding and complementary counts. It omits bare integers and text inside several formatting and link commands, and does not establish that a numerical match supports the surrounding claim.

Availability. The system source, task fixtures, per-run result records, generation scripts, pre-registrations and dated deviation record are archived with the project repository at commit 28961b2 and the manuscript branch built on it.

Author and AI disclosure

Author. [Maintainer name, affiliation, and contact to be filled before submission.]

AI disclosure. Language-model agents helped build the evaluated system, run its recorded experiments, and draft this manuscript. The human author is responsible for checking the evidence, citations, and final submitted text. No agent is listed as an author.

References

- [1] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- [2] Chuan He, Zebin Chen, Zhengyi Yang, Shaobo Qiao, Mingchen Ju, Jiate Liu, Dong Wen, and Guanfeng Liu. Minority sentinel: When to overturn majority voting in multi-agent llm debates, 2026. URL <https://arxiv.org/abs/2606.29270>.
- [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2023. URL <https://arxiv.org/abs/2310.06770>.
- [4] Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. Encouraging divergent thinking in large language models through multi-agent debate, 2023. URL <https://arxiv.org/abs/2305.19118>.
- [5] Freda Shi, Daniel Fried, Marjan Ghazvininejad, Luke Zettlemoyer, and Sida I. Wang. Natural language to code translation with execution, 2022. URL <https://arxiv.org/abs/2204.11454>.
- [6] Andries Smit, Paul Duckworth, Nathan Grinsztajn, Thomas D. Barrett, and Arnu Pretorius. Should we be going mad? a look at multi-agent debate strategies for llms, 2023. URL <https://arxiv.org/abs/2311.17371>.
- [7] Minsoo Song, Chanwoo Kim, Sugyeong Eo, and Chanjun Park. Beyond consensus: Downward bias and role asymmetry in multi-agent llm judges for subjective evaluation, 2026. URL <https://arxiv.org/abs/2608.30373>.
- [8] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- [9] Binwei Yao, Chao Shang, Wanyu Du, Jianfeng He, Ruixue Lian, Yi Zhang, Hang Su, Sandesh Swamy, and Yanjun Qi. Peacemaker or troublemaker: How sycophancy shapes multi-agent debate, 2025. URL <https://arxiv.org/abs/2509.23055>.
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023. URL <https://arxiv.org/abs/2306.05685>.